■ BrandPulse AI

Sentiment Analysis Platform

Comprehensive Project Question Bank

All questions are strictly derived from the BrandPulse AI codebase and architecture.

■ Total Questions	■■ Sections	■■ Architecture	■ Date
72	11	Microservices	Oct 2026

React 18 + Vite • Node.js + Express • Python FastAPI • Prisma ORM • Scikit-Learn • TF-IDF + ML Pipeline
SQLite / PostgreSQL • Docker Compose • Framer Motion • JWT Auth • Sentiment140 Dataset • ABSA + Emotion AI

■ Table of Contents

1.	Project Overview & Architecture	Q1–Q6 (6 Qs)
2.	ML Service — Sentiment Engine	Q7–Q16 (10 Qs)
3.	Text Preprocessing Pipeline	Q17–Q19 (3 Qs)
4.	Backend API & Business Logic	Q20–Q29 (10 Qs)
5.	Database Schema & Prisma ORM	Q30–Q36 (7 Qs)
6.	Frontend Architecture & API Service	Q37–Q43 (7 Qs)
7.	DevOps, Docker & Deployment	Q44–Q49 (6 Qs)
8.	API Reference & Route Architecture	Q50–Q55 (6 Qs)
9.	Security, Rate Limiting & Error Handling	Q56–Q61 (6 Qs)
10.	Frontend Features & Theme System	Q62–Q67 (6 Qs)
11.	Training Pipeline & Model Artifacts	Q68–Q72 (5 Qs)

■■ Section 1: Project Overview & Architecture

Q1 **What is BrandPulse AI and what is its primary purpose?**

Answer: BrandPulse AI is a production-ready brand intelligence and sentiment monitoring platform. Its primary purpose is to analyze public sentiment, monitor brand health, and automatically alert users when negative feedback spikes exceed defined thresholds. It combines a React 18 frontend, Node.js/Express backend, and a Python FastAPI ML microservice.

Note: *The platform is also known as 'Sentiment AI' in its repository and package naming.*

Q2 **How many microservices does the BrandPulse AI platform consist of, and what are they?**

Answer: The platform consists of 3 microservices: (1) React 18 + Vite frontend running on port 5173, (2) Node.js + Express backend API running on port 5000, and (3) Python FastAPI ML inference microservice running on port 8000.

Q3 **What dataset was used to train the sentiment models in this project, and how many records does it contain?**

Answer: The models were trained on the Sentiment140 dataset, which contains 1.6 million tweets. The dataset is a binary classification CSV with sentiment labels 0 (negative) and 4 (positive). The training script uses a stratified sample of 200,000 rows by default (SAMPLE_SIZE = 200_000) for faster development cycles.

```
# train.py SAMPLE_SIZE = 200_000 # Set to None to use full 1.6M rows DATA_PATH = BASE_DIR / 'data' / 'training.1600000.processed.noemoticon.csv' COLUMNS = ['sentiment', 'id', 'date', 'query', 'user', 'text']
```

Q4 **What ports do each of the three services use, and where are they configured?**

Answer: ML Service: port 8000 (uvicorn), Backend API: port 5000 (configured via PORT env variable in backend/.env), Frontend: port 5173 (Vite default). In Docker Compose, the frontend is served by Nginx on port 3000.

Q5 **What is the difference between the DATABASE_URL used in local development vs. Docker Compose?**

Answer: In local development, the DATABASE_URL points to a SQLite file: 'file:./prisma/prisma/dev.db'. In Docker Compose (production), it uses a PostgreSQL connection string: 'postgresql://sentiment_user:sentiment_secret@database:5432/sentiment_ai?schema=public'. Prisma ORM handles both via its database-agnostic schema.

```
# Local (backend/.env) DATABASE_URL=file:./prisma/prisma/dev.db # Docker Compose DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@database:5432/${POSTGRES_DB}?schema=public
```

Q6 **Describe the health check mechanism used for the ML service in Docker Compose.**

Answer: The Docker Compose ml-service health check runs: 'curl -f http://localhost:8000/health || exit 1' at 15-second intervals with a 5-second timeout and 5 retries. The /health endpoint returns {status: 'ok', service: 'ml-service', loaded_models: [...]}. The backend service declares 'condition: service_healthy' on the ml-service dependency.

```
healthcheck: test: ["CMD-SHELL", "curl -f http://localhost:8000/health || exit 1"] interval: 15s timeout: 5s retries: 5
```

■ Section 2: ML Service — Sentiment Engine

Q7 **What four machine learning models are supported by the ML microservice?**

Answer: The ML service supports four models: (1) Logistic Regression, (2) Linear SVC (Support Vector Classifier), (3) Multinomial Naive Bayes, and (4) XGBoost. All four are trained using a TF-IDF vectorization pipeline and stored as .joblib files in the artifacts/ directory.

```
MODEL_NAMES = {'logistic_regression', 'linear_svc', 'naive_bayes', 'xgboost'}
```

Q8 **What is the artifact loading priority order in the ML service's model registry?**

Answer: The ML service first tries to load .joblib pipeline files from the artifacts/ directory. If not found, it falls back to legacy .pkl files at the root level (e.g., logistic_regression_model.pkl + tfidf_vectorizer.pkl). When a legacy pkl is loaded and combined with the vectorizer into a Pipeline, it is automatically saved as a .joblib for future use.

Note: *The logistic_regression_model.pkl and tfidf_vectorizer.pkl at the root of ml-service/ are the original legacy artifacts that were pre-trained separately.*

Q9 **What is the fallback prediction mechanism when no trained model is available?**

Answer: The _fallback_predict() function uses a keyword heuristic approach. It splits the text into words and counts matches against two hardcoded sets: positive words (love, great, excellent, awesome, good, happy, recommend, best, perfect, fantastic, wonderful) and negative words (bad, terrible, worst, hate, awful, poor, broken, horrible, refund, slow, disappointing). If positive hits >= negative hits, it returns positive with 0.72 confidence; otherwise negative with 0.75 confidence.

```
pos_words = {'love','great','excellent','awesome','good','happy','recommend',
'best','perfect','fantastic','wonderful'} neg_words =
{'bad','terrible','worst','hate','awful','poor',
'broken','horrible','refund','slow','disappointing'}
```

Q10 **How does the ML service handle the Sentiment140 label encoding (0 vs 4) when computing probabilities?**

Answer: Sentiment140 uses label 4 for positive and 0 for negative. When predict_proba() is available, the service checks the pipeline's classes_ attribute. If class 4 is present, it maps pos_idx to classes.index(4) and neg_idx to classes.index(0). If classes contain {0, 1}, it uses standard binary mapping. For models with decision_function (like Linear SVC), it applies the sigmoid function: pos_prob = 1.0 / (1.0 + math.exp(-decision)).

```
is_pos = (raw_label == 1 or raw_label == 4) label = 1 if is_pos else 0 # Sigmoid for
LinearSVC: pos_prob = 1.0 / (1.0 + math.exp(-decision))
```

Q11 **What does the /intelligence endpoint return, and what three analysis pipelines does it run?**

Answer: The /intelligence endpoint runs three analysis pipelines: (1) Emotion detection via detect_emotion() — returns primary emotion + 8-class distribution, (2) Intent classification via classify_intent() — returns intent + 8-class distribution + confidence, (3) Aspect-Based Sentiment Analysis via extract_aspects() — returns up to 10 aspects with per-aspect sentiment and matched keywords. It also computes a priority_score (0.0–1.0) and risk_level ('low', 'medium', 'high', 'critical').

Q12 **How is the priority_score calculated in the /intelligence endpoint?**

Answer: Priority is computed additively: If intent is in {complaint, refund_request, urgency}: priority += 0.5 + intent_conf * 0.2. If emotion is in {anger, disgust, fear}: priority += emotion_dist[emotion] * 0.3. Each negative aspect found adds 0.05 up to a max of 0.2. The score is clamped to [0.0, 1.0]. Risk levels: critical ≥ 0.75, high ≥ 0.5, medium ≥ 0.25, else low.

```
if intent in URGENT_INTENTS: priority += 0.5 + intent_conf * 0.2 if emotion in URGENT_EMOTIONS: priority += emotion_dist.get(emotion, 0) * 0.3 neg_aspects = sum(1 for a in aspects if a.sentiment == 'NEGATIVE') priority += min(0.2, neg_aspects * 0.05) priority = round(min(1.0, priority), 3)
```

Q13 **What are the 10 predefined brand aspects tracked by the Aspect-Based Sentiment Analysis (ABSA) module?**

Answer: The 10 predefined aspects in aspects.py are: pricing, customer_service, quality, performance, usability, delivery, reliability, features, value_for_money, and brand_reputation. Each aspect has its own keyword list (15–18 keywords). The system uses keyword matching + contextual polarity cues with negation and intensifier handling.

Q14 **How does the ABSA module handle negation in polarity scoring?**

Answer: When a positive or negative cue word is found, the system extracts the 25 characters preceding it and checks for negation words (not, no, never, don't, doesn't, etc.). If a negation is found, the score is flipped (positive cue adds to neg_score, negative cue adds to pos_score). Intensifiers (very, extremely, incredibly, etc.) within the 25-char window add a 1.5x multiplier.

```
if pre_words & NEGATIONS: neg_score += multiplier # flip: positive cue under negation else: pos_score += multiplier multiplier = 1.5 if pre_words & INTENSIFIERS else 1.0
```

Q15 What are the 8 emotion classes detected by the emotion.py module?

Answer: The 8 emotion classes are: joy, anger, sadness, fear, surprise, disgust, trust, and anticipation. When no signals are detected, the system returns 'neutral' with a uniform probability distribution (1/8 = 0.125 per emotion). Each emotion has an associated lexicon of trigger words/phrases.

Note: *These correspond to Robert Plutchik's wheel of basic emotions.*

Q16 What is the model alias mapping in the _resolve_model_name() helper?

Answer: The _resolve_model_name() helper maps legacy frontend aliases to actual model names: 'bilstm' → 'logistic_regression', 'ml' → 'logistic_regression', 'cnn' → 'linear_svc'. This ensures backward compatibility with older frontend versions that used different model identifiers.

```
alias_map = { 'bilstm': 'logistic_regression', 'ml': 'logistic_regression', 'cnn':
'linear_svc', }
```

■ Section 3: Text Preprocessing Pipeline

Q17 **What preprocessing steps does clean_text() perform in preprocessing.py?**

Answer: clean_text() performs 5 steps in order: (1) Lowercasing all text, (2) Removing URLs (https://, http://, www.), (3) Removing @mentions and #hashtags, (4) Removing HTML tags (via <.*?> regex), (5) Stripping excess whitespace. The function returns an empty string if the input is not a string.

```
text = text.lower() text = re.sub(r'https?://\S+|www\.\S+', '', text) # URLs text = re.sub(r'[@#]\w+', '', text) # mentions/hashtags text = re.sub(r'<.*?>', '', text) # HTML tags text = re.sub(r'\s+', ' ', text).strip() # whitespace
```

Q18 **What does tokenize_and_lemmatize() return, and what library dependencies does it use?**

Answer: tokenize_and_lemmatize() returns a list of cleaned, lemmatized tokens with stopwords removed. It calls clean_text() first, removes punctuation using str.maketrans, then tokenizes by splitting on spaces. If NLTK is available, it uses WordNetLemmatizer and the English stopword list. If NLTK is unavailable (ImportError), it gracefully falls back to no lemmatization and an empty stopword set.

Q19 **In the ML inference pipeline, what happens when clean_text() returns an empty string?**

Answer: When clean_text() returns an empty string (e.g., the input was only URLs/mentions), the pipeline falls back to using the original raw text as input: 'input_text = cleaned if cleaned else text'. This prevents the model from receiving empty input which would otherwise cause a prediction error.

```
cleaned = clean_text(text) input_text = cleaned if cleaned else text
```

■ Section 4: Backend API & Business Logic

Q20 What security middleware does the Express backend use in app.js?

Answer: The backend uses 4 layers of security middleware: (1) Helmet — sets secure HTTP headers, (2) CORS — strict origin validation against allowed origins list, (3) Rate limiting — auth endpoints: 20 req/15min window; general API: 300 req/15min window, (4) Morgan — HTTP request logging (disabled in test mode).

```
authLimiter: windowMs=15*60*1000, limit=20 apiLimiter: windowMs=15*60*1000, limit=300
```

Q21 How does the CORS configuration determine whether to allow a request origin?

Answer: The CORS origin callback allows a request if: (1) The request has no origin header (server-to-server calls), (2) The origin is in the allowedOrigins array (read from FRONTEND_URL env variable, comma-separated), OR (3) The origin matches the regex pattern /^http://(localhost|127.0.0.1)(:\d+)?$/. If none match, a 'Origin not allowed by CORS' error is thrown and returned as a 403 response.

```
const isAllowed = allowedOrigins.includes(origin) ||
/^http:\/\/(localhost|127\.0\.0\.1)(:\d+)?$/.test(origin);
```

Q22 What is the maximum text length accepted by the POST /api/analyses endpoint?

Answer: The backend enforces a maximum of 10,000 characters per text submission. If text.length > 10000, the endpoint returns a 400 error: 'text must be ≤ 10 000 characters'. The ML service schema also enforces max_length=10000 via Pydantic Field validation as a second layer of defense.

Q23 What are the four allowed model names validated by the backend API?

Answer: The backend validates the model field against: ALLOWED_MODELS = ['logistic_regression', 'linear_svc', 'naive_bayes', 'xgboost']. Any other value returns a 400 error. The default model used when none is specified is 'logistic_regression'.

Q24 **How does the /api/analyses POST route call the intelligence pipeline, and what happens if it fails?**

Answer: After calling /predict for sentiment, the route calls /intelligence in a separate try/catch block with a 15-second timeout. If the intelligence call fails (network error, timeout, etc.), it logs a warning '[analysisRoutes] Intelligence call failed' but does NOT fail the entire request. Default values are used: emotion='neutral', intent='general_feedback', aspects='[]', priorityScore=0.0, riskLevel='low'. This makes intelligence data optional/non-blocking.

Note: *This is an important fault-tolerance design decision — sentiment analysis always works even if the intelligence pipeline is unavailable.*

Q25 **What HTTP status codes does the analysis route return for ML service failures?**

Answer: 503 Service Unavailable: when err.code === 'ECONNREFUSED' or 'ECONNABORTED' (ML service is down/unreachable). 502 Bad Gateway: when err.response is present (ML service returned an error response). 201 Created: on successful creation. 400 Bad Request: for validation errors. 204 No Content: for successful DELETE.

Q26 **Explain the batch CSV import feature: what are its constraints and how does it trigger alerts?**

Answer: CSV import (POST /api/reports/import) constraints: max file size 2MB, max 500 rows per batch, CSV must have a 'text' column, each text max 10,000 chars. Processing: rows are scored sequentially via the /predict ML endpoint. Alert trigger: if 50% or more of successfully imported rows are negative (negativeCount / imported >= 0.5), an Alert record is created with type='negative_spike'.

```
if imported > 0 and negativeCount / imported >= 0.5: await prisma.alert.create({ data: { userId, title: 'Negative sentiment spike detected', message: f'{negativeCount} of {imported} imported posts were negative.', type: 'negative_spike' } })
```

Q27 **What does the GET /api/analyses/summary endpoint compute and return?**

Answer: The summary endpoint fetches all analyses for the authenticated user (selecting only sentiment and confidence fields), then computes: total count, positive count, negative count, positivePercentage (rounded to 1 decimal), negativePercentage (rounded to 1 decimal), and avgConfidence (average of all non-null confidence values, expressed as a percentage rounded to 1 decimal). Returns 0 for all metrics if no analyses exist.

Q28 How is JWT authentication implemented in the authService?

Answer: JWT tokens are signed with jwt.sign({userId, email, role}, JWT_SECRET, {expiresIn: JWT_EXPIRES_IN}) using the jsonwebtoken library. Bcrypt with salt factor 10 is used for password hashing. Each login creates a Session record in the database storing the token, userAgent, ipAddress, and a hardcoded 7-day expiry. Signout deletes the Session record (token invalidation in DB, not just JWT expiry).

```
const salt = await bcrypt.genSalt(10); const passwordHash = await bcrypt.hash(password, salt); const token = jwt.sign( { userId: user.id, email: user.email, role: user.role }, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN } );
```

Q29 What avatar URL strategy does the signup flow use?

Answer: On signup, if no name is provided, the username defaults to the part of the email before '@'. The avatar URL is auto-generated using the DiceBear API with the 'bottts' style seeded by the URL-encoded email: 'https://api.dicebear.com/7.x/bottts/svg?seed={encoded_email}'. This ensures every user gets a unique, deterministic avatar.

```
const avatarUrl = `https://api.dicebear.com/7.x/bottts/svg?seed=${encodeURIComponent(normalizedEmail)}`;
```

■■ Section 5: Database Schema & Prisma ORM

Q30 **List all the Prisma models in the schema and their primary purpose.**

Answer: The schema defines 9 models: User (account & profile), Session (JWT session tracking), Brand (legacy global brand entity), Sentiment (legacy sentiment records), AspectMetric (per-brand aspect aggregations), Analysis (per-user analysis history — primary model), UserBrand (per-user brand keyword monitors), Alert (negative spike notifications), AIInsight (AI-generated insights), CustomerFeedback (human-in-the-loop label corrections), ModelVersion (model performance tracking).

Q31 **What is the data type used for IDs in the Prisma schema, and how are they generated?**

Answer: All primary keys use String type with @id @default(uuid()), meaning UUIDs are generated by the database. This is chosen over auto-incremented integers to support multi-tenant isolation and prevent ID enumeration attacks.

Q32 **How does the Analysis model store aspects and intelligence data?**

Answer: The Analysis model stores aspects as a JSON string in a String? field with @default('[]'). Similarly, emotion, intent, riskLevel, and priorityScore are stored as nullable fields with defaults. This is because SQLite does not support native JSON columns, so arrays/objects are serialized as JSON strings.

```
model Analysis { emotion String? @default('neutral') intent String?
@default('general_feedback') aspects String? @default('[]') // JSON string priorityScore
Float? @default(0.0) riskLevel String? @default('low') }
```

Q33 **What cascade behavior is defined for User deletion in the schema?**

Answer: When a User is deleted: Sessions are deleted (onDelete: Cascade), Analyses are deleted (Cascade), UserBrands are deleted (Cascade), Alerts are deleted (Cascade), AIInsights are deleted (Cascade), CustomerFeedback is deleted (Cascade). Sentiment records have onDelete: SetNull (userId becomes null, preserving historical data). Brand-level Sentiments cascade on Brand deletion.

Q34 **What is the ModelVersion model used for, and what performance metrics does it store?**

Answer: ModelVersion tracks ML model deployments per user. It stores: name, version string, accuracy, precision, recall, f1 (all nullable Float), datasetVersion (String), and status ('DRAFT', 'TESTING', 'PRODUCTION', 'ARCHIVED'). This supports model versioning, A/B testing, and rollback workflows.

Q35 **What unique constraints exist in the schema for preventing duplicate data?**

Answer: Three unique constraints: User.email is @unique (prevents duplicate accounts), Session.token is @unique (prevents token collision), UserBrand has @@unique([userId, name]) (a user cannot create two brand monitors with the same name), AspectMetric has @@unique([brandId, aspectName]) (one metric record per aspect per brand).

Q36 **What database indexes are defined in the schema and why?**

Answer: @@index([userId]) is defined on Session, Sentiment, Analysis, Alert, AIInsight, CustomerFeedback, and ModelVersion models. @@index([brandId]) on Sentiment and @@index([analysisId]) on CustomerFeedback. These indexes optimize the most common query pattern: fetching all records belonging to a specific user, which is the foundation of the multi-tenant data isolation pattern.

■ Section 6: Frontend Architecture & API Service

Q37 **How does apiService.js determine the backend API base URL?**

Answer: The BASE URL is determined from the Vite environment variable VITE_API_URL, defaulting to 'http://localhost:5000/api' if not set. Trailing slashes are stripped with .replace(/\/$/, ''). Similarly, ML_BASE uses VITE_ML_URL, defaulting to 'http://127.0.0.1:8000'.

```
const BASE = (import.meta.env.VITE_API_URL || 'http://localhost:5000/api').replace(/\/$/, ''); const ML_BASE = (import.meta.env.VITE_ML_URL || 'http://127.0.0.1:8000').replace(/\/$/, '');
```

Q38 **Describe the three-level fallback chain in the analyzeText() function.**

Answer: Level 1 (Primary): POST /api/analyses through the Express backend (authenticated, persisted to DB). Level 2 (Secondary fallback): If the backend fails, try POST directly to the ML microservice at port 8000/predict (unauthenticated, not persisted). Level 3 (Final fallback): If both fail, use a client-side keyword heuristic — same logic as the backend's _fallback_predict — and return a mock response without any API call.

Note: *This ensures the demo always works even when both backend and ML service are offline.*

Q39 **How is the JWT token stored and retrieved in the frontend?**

Answer: The JWT token is stored in localStorage under the key 'brandpulse_token'. The getToken() helper function retrieves it with localStorage.getItem('brandpulse_token'). It is attached as a Bearer token in the Authorization header of every request() call if present.

```
function getToken() { return localStorage.getItem('brandpulse_token'); } if (token) headers['Authorization'] = `Bearer ${token}`;
```

Q40 **How does the downloadReport() function implement file download without a separate download page?**

Answer: downloadReport() fetches the /reports/export endpoint, converts the response to a Blob, creates an object URL with URL.createObjectURL(blob), dynamically creates an anchor element, sets its href and download attribute to 'sentiment-report.csv', programmatically clicks it, then revokes the object URL. This triggers a browser file download without navigating away from the page.

```
const blob = await res.blob(); const url = URL.createObjectURL(blob); const a = document.createElement('a'); a.href = url; a.download = 'sentiment-report.csv'; a.click(); URL.revokeObjectURL(url);
```

Q41 **Why does importCsv() use a different fetch approach compared to the request() helper?**

Answer: importCsv() uses FormData and cannot use the standard request() helper because that helper sets 'Content-Type: application/json'. Multipart form uploads require the browser to set the Content-Type boundary automatically. Therefore, importCsv() bypasses the request() helper, only adds the Authorization header (not Content-Type), and sends the FormData body directly.

```
const form = new FormData(); form.append('file', file); form.append('model', model); // No 'Content-Type' header — browser sets multipart boundary automatically headers: token ? { Authorization: `Bearer ${token}` } : {}
```

Q42 **What frontend libraries/dependencies does the package.json specify?**

Answer: Runtime dependencies: react@^18.2.0, react-dom@^18.2.0, axios@^1.6.8, framer-motion@^13.4.4, lucide-react@^0.363.0. Dev dependencies: @vitejs/plugin-react@^4.2.1, vite@^5.2.0. The project does NOT use React Router — routing appears to be handled via state/conditional rendering or custom mechanisms.

Q43 **What is the Copilot feature in the API service, and which endpoint does it call?**

Answer: The sendCopilotMessage(message, history) function sends POST /api/copilot/chat with a {message, history} payload. This is an AI chat copilot feature that accepts a message string and a conversation history array, enabling multi-turn conversational AI within the dashboard. It is backed by the copilotService.js on the backend, which integrates with the Gemini AI API.

■ Section 7: DevOps, Docker & Deployment

Q44 What is the startup dependency order defined in docker-compose.yml?

Answer: The startup order is: 1) PostgreSQL database (no dependencies), 2) ML Service (no dependencies), 3) Express Backend (depends_on: database with condition: service_healthy AND ml-service with condition: service_healthy), 4) React Frontend (depends_on: backend — but without health condition). Both the database and ML service must pass health checks before the backend starts.

Q45 How are the ML model artifacts handled in Docker Compose deployment?

Answer: The ML service mounts a volume: './ml-service/artifacts:/app/artifacts'. This means the serialized model .joblib files are stored on the host machine (not baked into the Docker image) and mounted into the container at /app/artifacts. This allows model updates without rebuilding the Docker image.

```
volumes: - ./ml-service/artifacts:/app/artifacts
```

Q46 What PostgreSQL image and configuration does Docker Compose use?

Answer: The database service uses the postgres:16-alpine image. Configuration: POSTGRES_USER defaults to 'sentiment_user', POSTGRES_PASSWORD to 'sentiment_secret', POSTGRES_DB to 'sentiment_ai', port 5432. A named volume 'postgres_data' with local driver stores data. The health check uses pg_isready to confirm the database is accepting connections before dependent services start.

Q47 How is the frontend's Vite API URL configured during Docker build?

Answer: The VITE_API_URL is passed as a Docker build argument (args: VITE_API_URL: ${VITE_API_URL:-http://localhost:5000/api}). This is necessary because Vite embeds environment variables at build time (not runtime), so the URL must be known when building the static bundle. The frontend container serves the built static files via Nginx on port 80, exposed as port 3000.

```
frontend: build: args: VITE_API_URL: ${VITE_API_URL:-http://localhost:5000/api}
```

Q48 What does the /health endpoint on the backend return, and what is its purpose?

Answer: GET /health (not under /api prefix) returns: {status: 'ok', service: 'sentiment-ai-backend'}. It is an unauthenticated, unrate-limited endpoint used by load balancers, Docker Compose health checks, and monitoring tools to verify the backend process is running and responsive.

Q49 **What Docker network configuration is used and why?**

Answer: All services are connected to a custom bridge network named 'sentiment-network' (driver: bridge). This provides: (1) DNS-based service discovery (services can reach each other by name, e.g., 'http://ml-service:8000'), (2) Network isolation from other Docker networks, (3) No need to expose internal service ports (ML service uses expose: ['8000'] not ports, keeping it internal-only).

■ Section 8: API Reference & Route Architecture

Q50 **List all API route modules registered in the Express backend.**

Answer: 11 route modules are registered: authRoutes (/auth), analysisRoutes (/analyses), brandRoutes (/brands), alertRoutes (/alerts), reportRoutes (/reports), intelligenceRoutes (/intelligence), insightsRoutes (/insights), forecastRoutes (/forecast), copilotRoutes (/copilot), feedbackRoutes (/feedback), plus sentimentController routes. All are mounted under /api.

Q51 **Which API endpoints require authentication and which do not?**

Answer: No authentication required: POST /api/auth/signup, POST /api/auth/signin, GET /health, POST /predict (ML service direct). Authentication required (JWT Bearer token): GET /api/auth/me, POST /api/auth/signout, ALL /api/analyses routes, ALL /api/brands routes, ALL /api/alerts routes, ALL /api/reports routes, ALL /api/intelligence routes, ALL /api/insights routes, ALL /api/forecast routes, ALL /api/copilot routes, ALL /api/feedback routes.

Q52 **What is the response body format for the paginated GET /api/analyses endpoint?**

Answer: The paginated response has shape: {items: Analysis[], total: number, page: number, limit: number}. Page defaults to 1, limit defaults to 20 with a maximum of 100. Items are ordered by createdAt descending (newest first). The query uses Promise.all() for parallel count and data fetches for performance.

```
{ items: Analysis[], total: number, page: number, limit: number } // Pagination: skip = (page-1)*limit, take = limit
```

Q53 **What does the submitFeedback() API function do and what endpoint does it call?**

Answer: submitFeedback(analysisId, correctedLabel) calls POST /api/feedback with {analysisId, correctedLabel}. This implements a human-in-the-loop (HITL) label correction system. Users can correct a misclassified sentiment analysis, which creates a CustomerFeedback record linking the original Analysis to the corrected label. This data can be used for model retraining.

Q54 **What is the purpose of the Gemini AI integration in the backend, and what endpoints expose it?**

Answer: The Gemini AI integration (geminiService.js) provides AI-powered insights: executive summaries, recommendations, and root cause analysis. Exposed via: GET /api/insights/summary (executive summary), GET /api/insights/recommendations (AI recommendations), GET /api/insights/config (check Gemini config status), POST /api/insights/set-key (configure the Gemini API key at runtime). The GEMINI_API_KEY is stored in backend/.env.

Q55 **What does the forecast endpoint return, and how is it called from the frontend?**

Answer: getForecast(days = 30) calls GET /api/forecast?days={days}. The forecastService.js generates sentiment trend forecasts for the next N days based on historical analysis data. This feeds the dashboard's forecast/trend visualization charts. The default forecast window is 30 days.

■ Section 9: Security, Rate Limiting & Error Handling

Q56 **What is the rate limit configuration for authentication endpoints vs. general API endpoints?**

Answer: Auth endpoints (/api/auth/*): 20 requests per 15-minute window (brute-force protection for login/signup). General API (/api/*): 300 requests per 15-minute window. Both use standardHeaders: true (RateLimit-* headers) and legacyHeaders: false (no X-RateLimit-* headers). Exceeding limits returns: {error: 'Too many requests, please try again later.'}

Q57 **How does the backend handle CORS preflight and blocked origins?**

Answer: CORS errors for disallowed origins are caught in a dedicated error handler middleware: if (err.message === 'Origin not allowed by CORS') return res.status(403).json({error: 'Origin not allowed'}). This runs before the global errorHandler middleware. Standard CORS preflight (OPTIONS) requests are handled automatically by the cors() middleware.

```
app.use((err, _req, res, next) => { if (err.message === 'Origin not allowed by CORS') { return res.status(403).json({ error: 'Origin not allowed' }); } next(err); });
```

Q58 **Why does the authService return the same error message for both 'user not found' and 'wrong password'?**

Answer: Both conditions return 'Invalid email or password.' (with statusCode 401). This is intentional security hardening: if different messages were returned (e.g., 'Email not found' vs. 'Wrong password'), an attacker could use the error difference to enumerate valid email addresses through a user enumeration attack.

Q59 **What is the body size limit for the Express JSON parser?**

Answer: The Express JSON body parser is configured with limit: '1mb': app.use(express.json({limit: '1mb'})). Requests with JSON bodies larger than 1MB will be rejected with a 413 Payload Too Large error. This prevents large payload DoS attacks.

Q60 **What request timeout does the backend use when calling the ML service?**

Answer: For sentiment prediction (POST /predict): 30-second timeout (timeout: 30000). For intelligence pipeline (POST /intelligence): 15-second timeout (timeout: 15000). For CSV batch import rows: 30-second timeout per row. These are passed as axios options.

```
// Sentiment prediction await axios.post(`${ML_URL}/predict`, {...}, { timeout: 30000 }) // Intelligence pipeline await axios.post(`${ML_URL}/intelligence`, {...}, { timeout: 15000 })
```

Q61 **How does the CSV export endpoint prevent CSV injection attacks?**

Answer: All field values in the CSV export are wrapped with an esc() function: const esc = (v) => `"${String(v ?? '').replace(/"/g, '""')}"`; This double-quotes all values and escapes any embedded double-quotes by doubling them (CSV standard). However, it does not explicitly strip formula-injection characters like '=', '+', '-', '@' which some security scanners flag.

```
const esc = (v) => `"${String(v ?? '').replace(/"/g, '""')}"`;
```

■ Section 10: Frontend Features & Theme System

Q62 What are the 4 accent color palettes available in the BrandPulse AI frontend theme system?

Answer: The frontend ThemeContext provides 4 curated accent palettes: Violet, Blue, Emerald, and Rose. Combined with the Dark/Light/System mode switcher, users can personalize the UI. The theme settings are stored under user preferences in the Settings page.

Q63 What animation library does the frontend use for micro-animations?

Answer: The frontend uses Framer Motion (framer-motion@^13.4.4) for all micro-animations including page transitions, card animations, hover effects, and interactive elements. Lucide React (lucide-react@^0.363.0) provides the iconography system.

Q64 What are the main pages available in the user dashboard?

Answer: The dashboard has 8 main views: Overview (KPI cards + recent breakdown), Live Analyzer (real-time sentiment testing with model selection), Analytics View (distribution charts, ratios, volume), Batch Reports (drag-and-drop CSV upload), Analysis History (paginated table with search + filters), Alerts Center (negative spike notification feed), Settings & Preferences (profile, brand keywords, theme), and additional AI/Intelligence views.

Q65 What KPIs are shown on the Dashboard Overview page?

Answer: The Overview KPI cards display: Total Analyses, Positive % (positivePercentage), Negative % (negativePercentage), and Average Confidence (avgConfidence as percentage). These are fetched from GET /api/analyses/summary. The getSummary() function in apiService.js provides fallback mock data (total: 1420, positive: 980, negative: 440, positivePercentage: 69%, avgConfidence: 94%) if the API fails.

Q66 What data is included in the mock fallback for getHistory() in apiService.js?

Answer: The mock fallback returns 3 hardcoded items: (1) positive: 'The new M-series chip speed is unbelievable, builds finish in seconds! Absolutely loving it.' (conf: 0.96), (2) negative: 'App crashed during checkout, payment was deducted but order failed.' (conf: 0.94), (3) positive: 'Incredible storytelling, gorgeous cinematography, and a masterpiece soundtrack.' (conf: 0.98). All use 'logistic_regression' model.

Q67 What glassmorphism styling approach is used in the frontend?

Answer: The frontend uses glassmorphic card styling (backdrop-filter: blur + semi-transparent backgrounds) combined with Framer Motion animations, dark mode support, and the 4-palette accent system. The App.css file (122KB) contains the entire design system. The index.css provides minimal base resets.

■■ Section 11: Training Pipeline & Model Artifacts

Q68 What does train.py produce as output artifacts?

Answer: train.py produces 5 output files in ml-service/artifacts/: logistic_regression.joblib, linear_svc.joblib, naive_bayes.joblib, xgboost.joblib (all scikit-learn Pipelines with TF-IDF + classifier), and metrics.json (accuracy, precision, recall, F1 for each model).

Q69 What TF-IDF vectorizer configuration is used in the training pipeline?

Answer: The TF-IDF pipeline uses sklearn's TfidfVectorizer with n-gram range (1, 2) — unigrams and bigrams — which significantly improves feature coverage over single words. The vectorizer is fit on the training set and included in the Pipeline object so it is part of the same .joblib artifact as the classifier, ensuring consistent preprocessing at inference time.

Q70 How does the training script handle the XGBoost dependency?

Answer: XGBoost is imported inside a try/except block. If 'from xgboost import XGBClassifier' raises ImportError, HAS_XGBOOST is set to False and a warning is printed: 'WARNING: xgboost not installed — skipping XGBoost model.' The training proceeds for the other 3 models. This makes XGBoost an optional dependency rather than a hard requirement.

```
try: from xgboost import XGBClassifier HAS_XGBOOST = True except ImportError: HAS_XGBOOST = False print('WARNING: xgboost not installed — skipping XGBoost model.')
```

Q71 What Python packages are listed in ml-service/requirements.txt?

Answer: The 10 packages are: fastapi==0.110.0, uvicorn[standard]==0.29.0, pydantic==2.6.4, pydantic-settings==2.2.1, numpy==1.26.4, pandas==2.2.1, scikit-learn==1.4.1.post1, xgboost==2.0.3, joblib==1.3.2, python-multipart==0.0.9, requests==2.31.0. (Note: NLTK for preprocessing is a soft dependency — handled with try/except in preprocessing.py.)

Q72 What is the file path the training script expects for the Sentiment140 dataset?

Answer: The training script expects the dataset at: ml-service/data/training.1600000.processed.noemoticon.csv. It uses encoding='latin-1' (not UTF-8) to read the file, and no header row — column names are manually assigned via COLUMNS = ['sentiment', 'id', 'date', 'query', 'user', 'text']. If the file doesn't exist, a detailed FileNotFoundError with the Kaggle download URL is raised.

```
DATA_PATH = BASE_DIR / 'data' / 'training.1600000.processed.noemoticon.csv' df = pd.read_csv(DATA_PATH, encoding='latin-1', header=None, names=COLUMNS)
```

This question bank is generated exclusively from the BrandPulse AI / Sentiment AI project source code, schema definitions, route handlers, ML pipeline, and Docker orchestration configuration. All answers reference actual implementation details from the codebase.